

Positive testing: Unary operations

Test 1 — GetMovieDetail (positive)

Endpoint: ws://localhost:8080/ws/movies/detail

Request sent:

```
{"movieId": 1}
```

Expected response: The server returns a MovieDetailResponse JSON object containing the full movie details for Inception, including a genres array.

```
{
  "id": 1,
  "title": "Inception",
  "releaseYear": 2010,
  "runtimeMinutes": 148,
  "director": "Christopher Nolan",
  "synopsis": "...",
  "genres": ["Action", "Sci-Fi"]
}
```

Result:

The screenshot shows a websocket testing tool interface. At the top, the URL bar displays 'ws://localhost:8080/ws/movies/detail'. Below the URL bar, there are tabs for 'Docs', 'Message', 'Params', 'Headers', and 'Settings'. The 'Message' tab is active, showing a message box with the text '{"movieId": 1}' and a 'Send' button. To the right of the message box, there is a 'Disconnect' button. Below the message box, there is a 'Response' section. The 'Response' section shows a list of messages. The first message is '{"director": "Christopher Nolan", "genres": ["Action", "Sci-Fi"], "id": 1, "releaseYear": 2010, "r...' with a timestamp of '00:25:38.123'. The second message is '{"movieId": 1}' with a timestamp of '00:25:38.118'. There is a 'Restore' button at the bottom right of the response section. The interface also includes a 'Save' button, a 'Share' button, and a 'Cookies' section on the right side.

Negative testing: Unary operations

Test 2 — GetMovieDetail (malformed JSON)

Endpoint: `ws://localhost:8080/ws/movies/detail`

Request sent:

`abc`

Expected response: A structured JSON error envelope:

```
{"error": "invalid_request", "message": "Malformed JSON: expected {\"movieId\": <integer>}"}
```

The connection stays open — the client can send another valid request on the same connection.

Result:

The screenshot shows a websocket testing interface. At the top, the URL `ws://localhost:8080/ws/movies/detail` is entered. Below the input field, the 'Message' tab is selected, showing the sent message `abc`. The 'Response' tab is also visible, showing a list of messages. The first message in the list is a JSON error response: `{"error": "invalid_request", "message": "Malformed JSON: expected {\"movieId\": <integer>}"}`, received at `00:27:18.184`. The second message is `abc`, received at `00:27:18.180`. The third message is `Connected to ws://localhost:8080/ws/movies/detail`, received at `00:27:17.212`. The interface also includes a 'Send' button, a 'Disconnect' button, and a 'Cookies' section on the right.

Test 3 — GetMovieDetail (not found)

Endpoint: ws://localhost:8080/ws/movies/detail

Request sent:

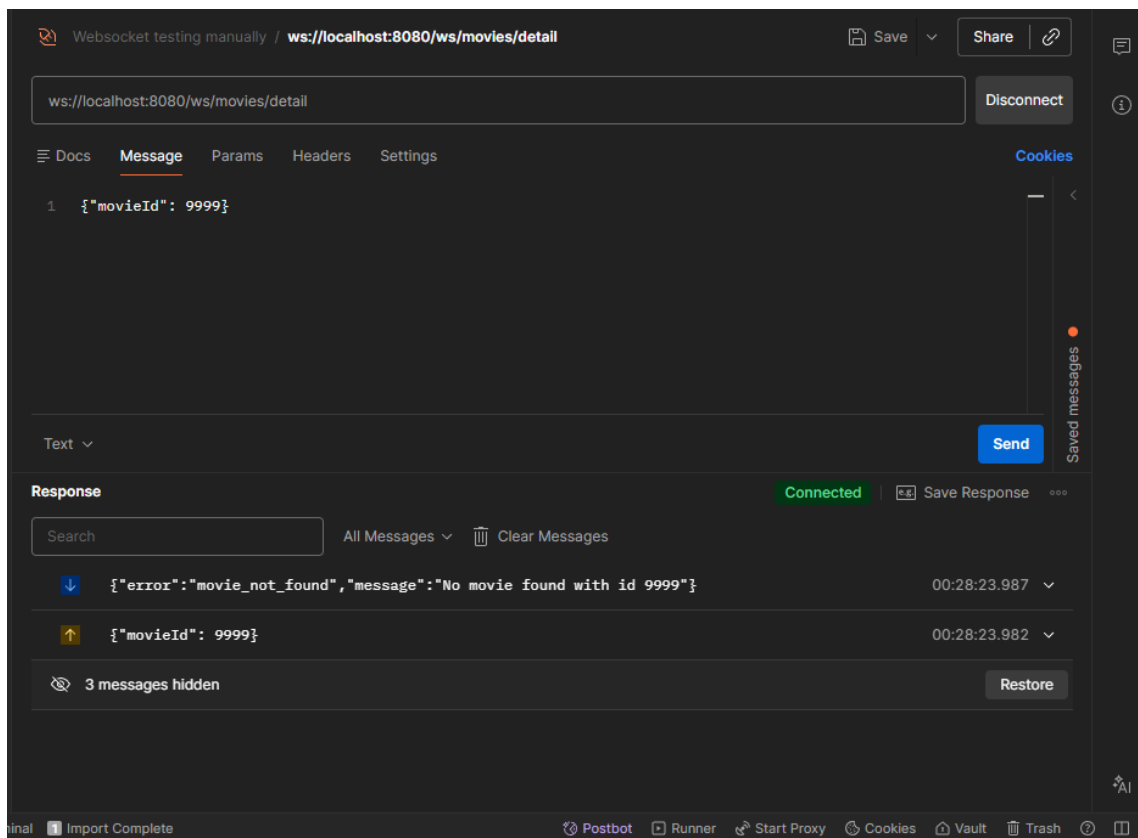
```
{"movieId": 9999}
```

Expected response: A structured JSON error envelope:

```
{"error": "movie_not_found", "message": "No movie found with id 9999"}
```

The connection stays open.

Result:



Positive testing: Bidirectional Streaming operations

Test 4 — ReviewStream (subscribe and receive reviews)

Endpoint: ws://localhost:8080/ws/movies/reviews/stream

Request sent:

```
{"movieId": 1}
```

Expected response: The server immediately streams all existing reviews for movie 1 as individual ReviewResponse JSON frames. Each frame contains reviewId, movieId, movieTitle, rating, comment, and createdAt.

Movie 1 (Inception) has 4 reviews in the database (ids 101, 102, 103, 111), so we expect 4 frames. Example first frame:

```
{
  "reviewId": 101,
  "movieId": 1,
  "movieTitle": "Inception",
  "rating": 9,
  "comment": "A masterpiece of layered storytelling.",
  "createdAt": "..."}
}
```

The connection stays open. The server continues polling every ~2 seconds for new reviews.

Result:

The screenshot shows a web browser interface for testing websockets. The address bar displays the websocket URL: `ws://localhost:8080/ws/movies/reviews/stream`. Below the address bar, there are tabs for 'Docs', 'Message', 'Params', 'Headers', and 'Settings'. The 'Message' tab is active, showing the request: `1 {"movieId": 1}`. To the right of the message input, there is a 'Send' button. Below the message input, there is a 'Response' section. The 'Response' section shows a list of received messages. The first message is a 'Connected' message: `Connected to ws://localhost:8080/ws/movies/reviews/stream`. The subsequent messages are review frames for movie 1, each containing a reviewId, movieId, movieTitle, rating, comment, and createdAt. The messages are: `1 {"comment": "A masterpiece of layered storytelling.", "createdAt": "2026-06-01T22:23:16", "movieId": 1, "movieTitle": "Inception", "rating": 9, "reviewId": 101}`, `2 {"comment": "Ambitious but the ending is too ambiguous.", "createdAt": "2026-06-01T22:23:16", "movieId": 1, "movieTitle": "Inception", "rating": 8, "reviewId": 102}`, `3 {"comment": "Rewatchable every single year.", "createdAt": "2026-06-01T22:23:16", "movieId": 1, "movieTitle": "Inception", "rating": 9, "reviewId": 103}`, and `4 {"comment": "A masterpiece of layered storytelling.", "createdAt": "2026-06-01T22:23:16", "movieId": 1, "movieTitle": "Inception", "rating": 9, "reviewId": 111}`. The interface also includes a 'Disconnect' button, a 'Save' button, and a 'Share' button. The bottom of the browser shows various extensions and the status bar.

Test 5 — ReviewStream (add second subscription on same connection)

Endpoint: `ws://localhost:8080/ws/movies/reviews/stream`

Request sent (on same connection):

```
{"movieId": 2}
```

Expected response: The server streams all existing reviews for movie 2 (The Dark Knight) on the same connection. Movie 2 has 3 reviews in the database (ids 104, 105, 106), so we expect 3 additional frames. Both movie 1 and movie 2 reviews now flow on this connection.

Result:

The screenshot shows a websocket testing interface with the following details:

- URL:** `ws://localhost:8080/ws/movies/reviews/stream`
- Message:** `1 {"movieId": 2}`
- Status:** Connected (indicated by a green dot)
- Response Messages:**
 - `{"comment": "Excellent, though the third act drags slightly.", "createdAt": "2026-06-01T22:23:16", "movieId": 2, "movieTitle": "The Dark Knight"}` (11:42:56.553)
 - `{"comment": "A crime thriller that happens to feature Batman.", "createdAt": "2026-06-01T22:23:16", "movieId": 2, "movieTitle": "The Dark Knight"}` (11:42:56.552)
 - `{"comment": "Best superhero movie ever made.", "createdAt": "2026-06-01T22:23:16", "movieId": 2, "movieTitle": "The Dark Knight"}` (11:42:56.551)
 - `{"movieId": 2}` (11:42:56.542)
 - `{"comment": "<script>alert('xss');</script>", "createdAt": "2026-06-01T22:23:16", "movieId": 1, "movieTitle": "Inception"}` (11:41:50.495)
 - `{"comment": "Rewatchable every single year.", "createdAt": "2026-06-01T22:23:16", "movieId": 1, "movieTitle": "Inception"}` (11:41:50.495)
 - `{"comment": "Ambitious but the ending is too ambiguous.", "createdAt": "2026-06-01T22:23:16", "movieId": 1, "movieTitle": "Inception"}` (11:41:50.494)
 - `{"comment": "A masterpiece of layered storytelling.", "createdAt": "2026-06-01T22:23:16", "movieId": 1, "movieTitle": "Inception"}` (11:41:50.493)
 - `{"movieId": 1}` (11:41:50.408)

Test 6 — ReviewStream (live push — add review while subscribed)

Endpoint: `ws://localhost:8080/ws/movies/reviews/stream`

While the stream connection is open and subscribed to movie 1, a new review is added

Expected response: Within ~2 seconds, the newly added review appears as a `ReviewResponse` frame on the WebSocket connection — without the client sending any message.

Result:

The screenshot shows a WebSocket testing interface with the URL `ws://localhost:8080/ws/movies/reviews/stream`. The interface includes tabs for Docs, Message, Params, Headers, and Settings. A message is sent: `1 {"movieId": 2}`. The response section shows a list of messages with a search bar and "All Messages" and "Clear Messages" buttons. The messages are as follows:

Direction	Message	Timestamp	Action
Down	<code>{"comment": "TESTING NEW ONE", "createdAt": "2026-06-03T09:44:32", "movieId": 1, "movieTitle": "Inception", "rating": ...}</code>	11:44:33.340	Down Arrow
Down	<code>{"comment": "Added during live stream test", "createdAt": "2026-06-03T08:57:13", "movieId": 2, "movieTitle": "The D..."}</code>	11:42:56.553	Down Arrow
Down	<code>{"comment": "Excellent, though the third act drags slightly.", "createdAt": "2026-06-01T22:23:16", "movieId": 2, "..."}</code>	11:42:56.553	Down Arrow
Down	<code>{"comment": "A crime thriller that happens to feature Batman.", "createdAt": "2026-06-01T22:23:16", "movieId": 2, "..."}</code>	11:42:56.552	Down Arrow
Down	<code>{"comment": "Best superhero movie ever made.", "createdAt": "2026-06-01T22:23:16", "movieId": 2, "movieTitle": "The..."}</code>	11:42:56.551	Down Arrow
Up	<code>{"movieId": 2}</code>	11:42:56.542	Up Arrow
Down	<code>{"comment": "<script>alert(&#39;xss&#39;)&lt;/script>", "createdAt": "2026-06-01T22:23:16", "movieId": 1, "..."}</code>	11:41:50.495	Down Arrow
Down	<code>{"comment": "Rewatchable every single year.", "createdAt": "2026-06-01T22:23:16", "movieId": 1, "movieTitle": "Ince..."}</code>	11:41:50.495	Down Arrow
Down	<code>{"comment": "Ambitious but the ending is too ambiguous.", "createdAt": "2026-06-01T22:23:16", "movieId": 1, "movie..."}</code>	11:41:50.494	Down Arrow

The interface also shows a "Connected" status, a "Save Response" button, and a "Send" button. The bottom bar includes options like "Find and replace", "Console", "Terminal", "Import Complete", "Postbot", "Runner", "Start Proxy", "Cookies", "Vault", "Trash", and "AI".

Negative testing: Bidirectional Streaming operations

Test 7 — ReviewStream (malformed JSON)

Endpoint: `ws://localhost:8080/ws/movies/reviews/stream`

Request sent:

`abc`

Expected response: A structured JSON error envelope:

```
{"error": "invalid_request", "message": "Malformed JSON: expected {\"movieId\": <integer>}"}
```

The connection stays open — the client can send a valid subscription on the same connection.

Result:

The screenshot shows a websocket testing interface. The URL bar contains `ws://localhost:8080/ws/movies/reviews/stream`. The 'Message' tab is active, showing a single message '1 abc' sent. The 'Response' tab is also active, showing a list of messages. The first message in the response list is a JSON error envelope: `{"error": "invalid_request", "message": "Malformed JSON: expected {\"movieId\": <integer>}"}`, received at 11:45:19.463. Subsequent messages in the list are valid JSON objects, including a comment and a movie ID update.

Direction	Message	Timestamp
Down	<code>{"error": "invalid_request", "message": "Malformed JSON: expected {\"movieId\": <integer>}"}</code>	11:45:19.463
Up	<code>abc</code>	11:45:19.425
Down	<code>{"comment": "TESTING NEW ONE", "createdAt": "2026-06-03T09:44:32", "movieId": 1, "movieTitle": "Inception", "rating": ...}</code>	11:44:33.340
Down	<code>{"comment": "Added during live stream test", "createdAt": "2026-06-03T08:57:13", "movieId": 2, "movieTitle": "The D..."}</code>	11:42:56.553
Down	<code>{"comment": "Excellent, though the third act drags slightly.", "createdAt": "2026-06-01T22:23:16", "movieId": 2, "..."}</code>	11:42:56.553
Down	<code>{"comment": "A crime thriller that happens to feature Batman.", "createdAt": "2026-06-01T22:23:16", "movieId": 2, "..."}</code>	11:42:56.552
Down	<code>{"comment": "Best superhero movie ever made.", "createdAt": "2026-06-01T22:23:16", "movieId": 2, "movieTitle": "The..."}</code>	11:42:56.551
Up	<code>{"movieId": 2}</code>	11:42:56.542

Test 8 — ReviewStream (not found)

Endpoint: ws://localhost:8080/ws/movies/reviews/stream

Request sent:

```
{"movieId": 9999}
```

Expected response: A structured JSON error envelope:

```
{"error": "movie_not_found", "message": "No movie found with id 9999"}
```

The connection stays open.

Result:

The screenshot shows a web browser window with a title bar that reads "Websocket testing manually / reviews bidirectional stream". The address bar contains the URL "ws://localhost:8080/ws/movies/reviews/stream". Below the address bar, there are tabs for "Docs", "Message", "Params", "Headers", and "Settings". The "Message" tab is selected, and it displays a single message: "1 {\"movieId\": 9999}\"". To the right of the message tab, there is a "Cookies" tab and a "Disconnect" button. Below the message tab, there is a "Text" dropdown menu and a "Send" button. The "Response" section is visible, showing a "Connected" status and a "Save Response" button. The response list contains two messages: the first is a JSON error object {"error": "movie_not_found", "message": "No movie found with id 9999"} received at 11:46:21.255, and the second is the request object {"movieId": 9999} received at 11:46:21.249. At the bottom, there is a message "14 messages hidden" and a "Restore" button.